Kingdom of Saudi Arabia

Ministry of Education

Prince Sattam Bin Abdulaziz University

College of Computer

Engineering and sciences

Dept of Software Engineering

المملكة العربية السعودية

وزارة التعليم

جامعة الأمير سطام بن عبد العزيز

كلية هندسة وعلوم الحاسب

قسم هندسة البرمجيات

SE 4222- Software maintenance & evaluation

Online used books platform

Shahad Abdullah Alnoaisser - 441051251

Ashwaq Rajeh Alotaibi - 441051382

Rema Mohammed Aldawood - 441051670

Danah khalid Almasaud - 441051716

Table of Contents

Table of Contents

Introduction....3
Problem Statement....3
Proposed Solution....3
Background....3
Project Description before the change:....4
Functional Requirements:....4
Non-functional requirements:....5
Diagrams:....6
- Class diagram:....6
- Use case:....6
Change request:....7
Change request schema:....7
Change implementation:....8
- Book listing:....8
- Search page:....13
Testing:....17
Project Description after the change:....20
Functional Requirements....20
Non-functional requirements :....21
Diagrams:....21
- Class diagram:....21
- Use case :....21

Introduction

The online used books platform is a web-based platform that facilitates the buying and selling of used books. The platform provides users with the opportunity to register, list books for sale, and browse and purchase books from other sellers. It aims to create a seamless and convenient experience for book enthusiasts to access a wide range of used books at affordable prices.

Problem Statement

The main problem faced by users of the used books platform is the lack of a streamlined and efficient process for buying and selling books. Buyers may struggle to find specific books or face difficulties in the approval process from sellers. Sellers may encounter challenges in pricing their books accurately and managing customer requests effectively. Additionally, both buyers and sellers may experience concerns related to trust, security, and quality assurance.

Proposed Solution

To address the aforementioned problems, a proposed solution is to develop and implement a user-friendly online platform that simplifies the process of buying and selling used books. The platform will offer a comprehensive registration system to authenticate users. For buyers, the purchase process will be simplified through an intuitive search function allowing them to easily find books by title or author. Sellers will have the responsibility of setting book prices, providing accurate book details, and managing customer requests efficiently. To ensure a seamless experience, the platform will handle the logistics of receiving, packaging, and shipping the books to customers. By centralizing the buying and selling process, the platform will offer a convenient, reliable, and cost-effective solution for individuals looking to engage in the used book market.

Background

The concept of online marketplaces for buying and selling used goods has gained significant popularity in recent years. E-commerce platforms have revolutionized the way people purchase products, and used books are no exception. Online used books platforms have emerged as a convenient alternative to physical stores, offering a vast selection of titles, competitive prices, and the convenience of home delivery.

The success of online marketplaces, such as Amazon and eBay, has paved the way for specialized platforms dedicated to the trade of used books. These platforms leverage technology to connect buyers and sellers, enabling them to easily navigate through extensive catalogs, communicate, and complete transactions securely. The growth of such platforms has contributed to the accessibility and affordability of used books, while also fostering a sense of community among book enthusiasts.

The platform's goal is to create a robust and reliable online marketplace where individuals can conveniently buy and sell used books, fostering a sustainable and eco-friendly approach to reading and literary exchange.

Project Description before the change:

Functional Requirements:

1. Register:
   - REQ-1: The user shall enter their email.
   - REQ-2: The user shall enter their username.
   - REQ-3: The user shall enter their phone number.
   - REQ-4: The system shall validate that the entered email and phone number are not already registered.
   - REQ-5: The user shall enter a password.

2. Log in:
   - REQ-1: The user shall enter their email or username.
   - REQ-2: The user shall enter their password.
   - REQ-3: The user shall be able to request a password reset.

3. Searching for a book:
   - REQ-1: The user shall enter the title of the book in the search bar.
   - REQ-2: The system shall display search results based on the entered title.
   - REQ-3: If there are no exact matches, the system shall display a notification informing the user.

4. Buying a book:
   - REQ-1: The user shall select the book(s) they want to buy.
   - REQ-2: The system shall show the buyer of the seller's approval or rejection of the purchase process.

5. Payment:
   - REQ-1: The user shall enter their location for shipping.
   - REQ-2: The user shall enter their credit card information for payment.

6. Displaying a book for sale:
   - REQ-1: The user shall enter the title of the book.
   - REQ-2: The user shall upload a photo of the book.
   - REQ-3: The user shall enter the price of the book.

7. Purchase order status:
   - REQ-1: The system shall send a notification to the buyer's email when someone requests to buy the book.
   - REQ-2: The seller shall go to the platform to determine the status of the request.

Non-functional requirements:

Usability:
NFR-1: The platform shall have a user-friendly and intuitive interface to ensure ease of use for all users, including those with limited technical expertise.

Performance:
NFR-1: The platform shall be highly responsive, to provide a seamless browsing and purchasing experience for users.
NFR-2: The system shall handle a large volume of book listings and user interactions efficiently, maintaining fast response times.

Security:
NFR-1: The platform shall employ robust security measures to protect user data, including personal information, payment details.

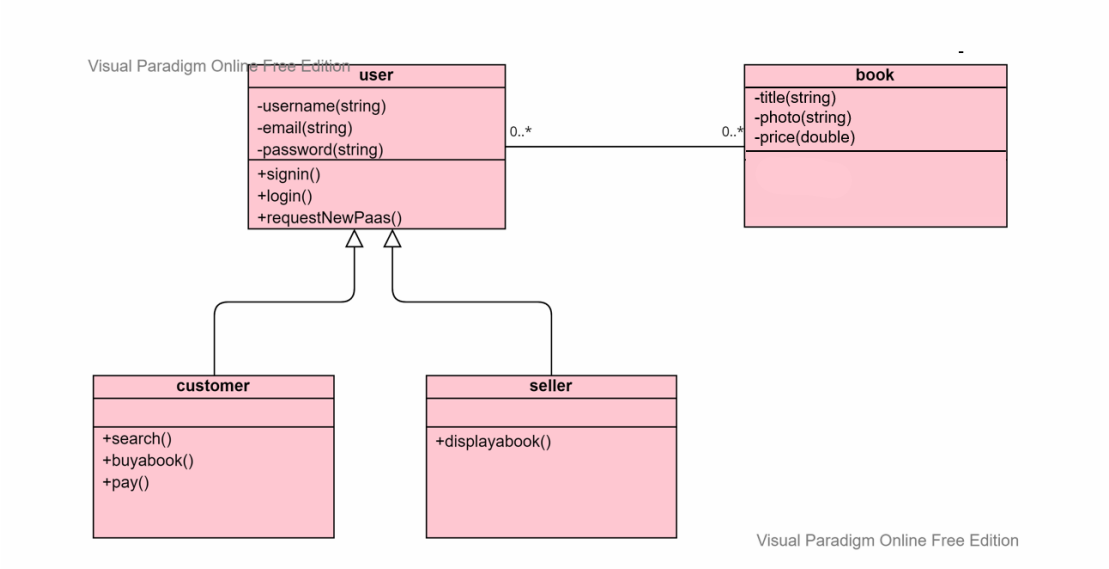
Reliability:
NFR-1: The platform shall have a high level of availability, minimizing downtime and service interruptions.
NFR-2: The system shall have data backup and recovery mechanisms in place to prevent data loss and ensure the integrity of user information.
NFR-3: The platform shall be able to handle and recover from errors or system failures gracefully, minimizing disruption to user activities.

Scalability:
NFR-1: The platform shall be designed to handle increasing numbers of users, books, and transactions as the user base and inventory grow over time.

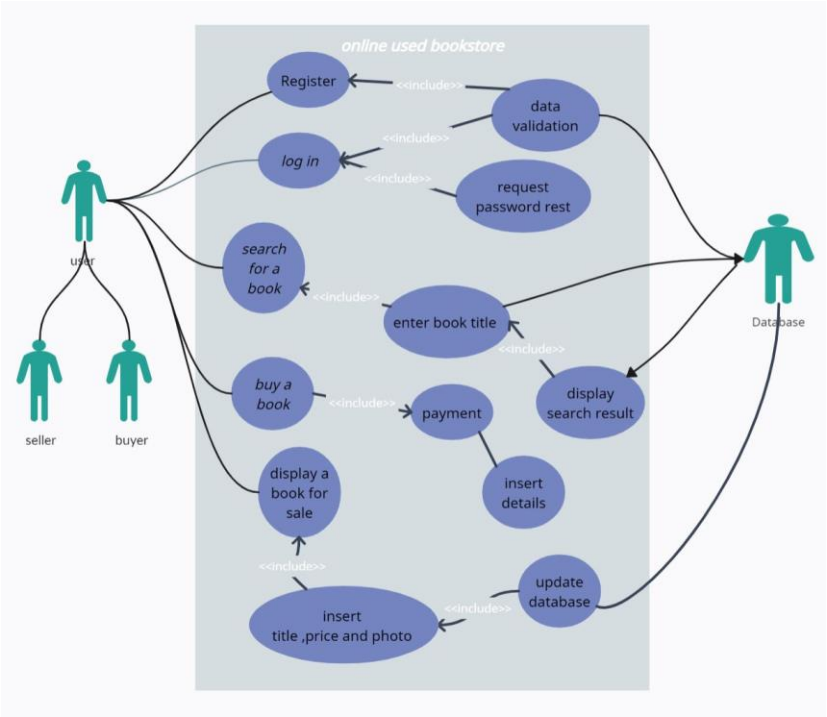
Integration:
NFR-1: The platform shall integrate with popular payment gateways to facilitate secure and convenient payment processing for users.
NFR-2: The system shall integrate with shipping and logistics services to provide accurate shipping costs, tracking information, and timely delivery of books to buyers.

Diagrams:

- Class diagram:



- Use case:



Change request:

Change request schema:

Field name	Value
Change_request_id	CR-001
Title	Update Book Listing and Search Features
Description	This change request aims to update the book listing functionality and the search feature to accommodate the additional fields added for book information. The book listing feature will be enhanced to include fields such as description, condition, and author, while the search feature will consider fields (description, author) when retrieving search results.
Maintenance type	Perfective
Product	Online used Books Platform
Component	Book Listing and Search Features
State	Submit
Customer	Project Team
Problem_origin	Enhancement Request
Impacts	Book Listing, Search Functionality
Resolution	The book listing functionality and the search feature will be updated to incorporate the new fields for book information. This will provide more comprehensive book details to buyers and improve their search experience. The book listing and search features will work together to ensure accurate and relevant search results based on the added fields
Note	None
Software_release	3.0.1
Committed_release	3.1.0
Priority	Normal
Severity	Normal
Marketing-justification	The updates to the book listing and search features will significantly enhance the user experience by providing detailed book information and improving search accuracy. This improvement will strengthen the platform's competitiveness in the market.
Time to implement	2 weeks per person
Eng assigned	Shahad Abdullah
Functional_spec_title	Book Listing and Search Features Functional Specification
Functional_spec_name	book_listing_spec.docx
Functional_spec_version	2.2
Decline_note	N/A
Ec_number	EC-123
Attachment	None
Tc_id	TC-001, TC-002, TC-003
Tc_results	Passed
Verification_method	Analysis, testing
Verification_status	Passed
Compliance	Compliance
Testing.note	Test cases executed successfully. No issues found during testing
Defect_id	N/A

Change implementation:

- Book listing:

- The code and the web page (see figure1) before the change:

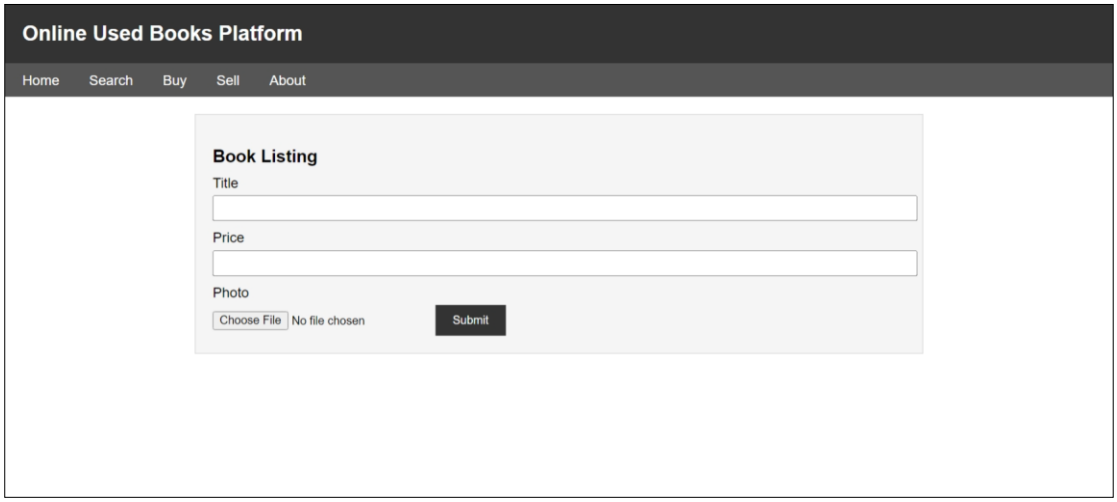


Figure 1: book listing page before change

```
<?php
if($_SERVER['REQUEST_METHOD'] === 'POST'){
$title=$_POST['title'];
$price=$_POST['price'];
$host='localhost';
$username='root';
$password='';
$database='book';
$conn=new mysqli($host,$username,$password,$database);
if($conn->connect_error){
die('Connection failed: '.$conn->connect_error);}
$sql="INSERT INTO books (title, price, photo_url) VALUES (?, ?, ?)";
$stmt=$conn->prepare($sql);
$stmt->bind_param("sds", $title, $price, $photoUrl);
if(isset($_FILES['photo'])&&$_FILES['photo']['error']===UPLOAD_ERR_OK){
$fileTmpPath=$_FILES['photo']['tmp_name'];
$fileName=$_FILES['photo']['name'];
$fileSize=$_FILES['photo']['size'];
$fileType=$_FILES['photo']['type'];
$uploadDir='uploads/';
$uploadFile=$uploadDir.uniqid().'_'.$fileName;
if(move_uploaded_file($fileTmpPath,$uploadFile)){
$photoUrl=$uploadFile;}else{
echo'Error uploading the file.';
exit();}}else{
$photoUrl=null;}
if($stmt->execute()){
echo'Book listing created successfully.';}else{
echo'Error creating book listing: '.$stmt->error;}
$stmt->close();
```

```
$conn->close();}
?>
<!DOCTYPE html>
<html>
<head>
  <title>Online Used Books Platform</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;       }
    header {
      background-color: #333;
      color: #fff;
      padding: 20px;       }
    h1 {
      margin: 0;
      font-size: 24px;       }
    nav {
      background-color: #555;
      padding: 10px;       }
    nav ul {
      list-style-type: none;
      margin: 0;
      padding: 0;       }
    nav ul li {
      display: inline;
      margin-right: 10px;       }
    nav ul li a {
      color: #fff;
      text-decoration: none;
      padding: 5px 10px;       }
    nav ul li a:hover {
      background-color: #777;       }
    main {
      max-width: 800px;
      margin: 20px auto;
      padding: 20px;
      background-color: #f5f5f5;
      border: 1px solid #ccc;       }
    h2 {
      font-size: 20px;
      margin-bottom: 10px;       }
    label {
      display: block;
      margin-bottom: 5px;       }
    input[type="text"],
    textarea {
      width: 100%;
      padding: 5px;
      margin-bottom: 10px;       }
```

```
      button[type="submit"] {
         padding: 10px 20px;
         background-color: #333;
         color: #fff;
         border: none;
         cursor: pointer;       }
      button[type="submit"]:hover {
         background-color: #555;       }
</style></head><body><header>
      <h1>Online Used Books Platform</h1>
   </header>
   <nav>
      <ul>
            <li><a href="index.html">Home</a></li>
            <li><a href="search.html">Search</a></li>
            <li><a href="buy.html">Buy</a></li>
            <li><a href="sell.html">Sell</a></li>
            <li><a href="about.html">About</a></li>
      </ul>
   </nav>
   <main>
      <h2>Book Listing</h2>
      <form method="POST" enctype="multipart/form-data">
         <label for="title">Title</label>
         <input type="text" id="title" name="title" required>

         <label for="price">Price</label>
         <input type="text" id="price" name="price" required>

         <label for="photo">Photo</label>
         <input type="file" id="photo" name="photo" accept="image/*" required>
         <button type="submit">Submit</button>
      </form>
   </main></body></html>
```

- The code and the web page (see figure 2) after the change:

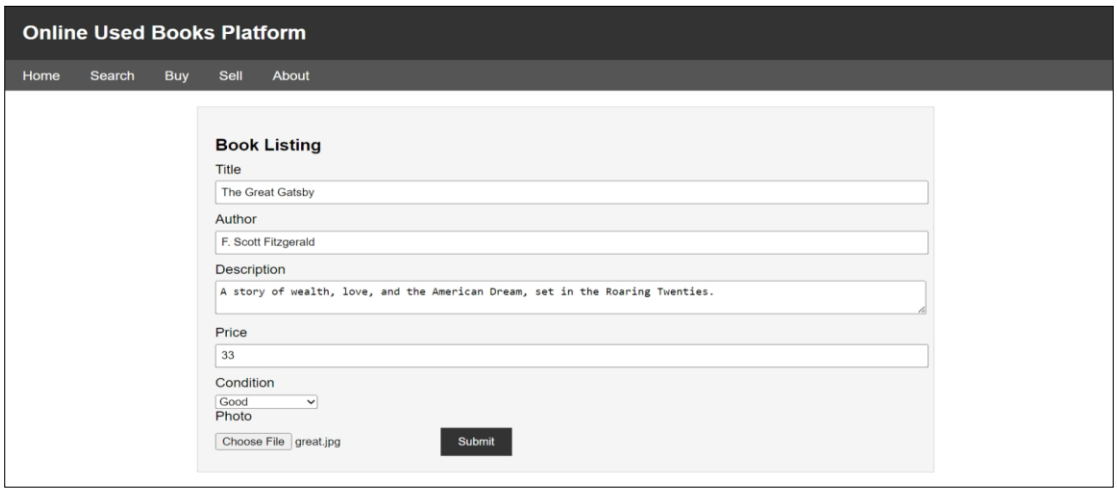


Figure 2: book listing page after change

```
<?php
if($_SERVER['REQUEST_METHOD'] === 'POST'){
$title=$_POST['title'];
$author=$_POST['author'];
$description=$_POST['description'];
$price=$_POST['price'];
$condition=$_POST['condition1'];
$host='localhost';
$username='root';
$password='';
$database='book';
$conn=new mysqli($host,$username,$password,$database);
if($conn->connect_error){
die('Connection failed: '.$conn->connect_error);}
$sql="INSERT INTO books (title,author,description,price,condition1,photo_url) VALUES
(?,?,?,?,?,?)";
$stmt=$conn->prepare($sql);
$stmt->bind_param("ssssss",$title,$author,$description,$price,$condition,$photoUrl);
if(isset($_FILES['photo'])&&$_FILES['photo']['error']===UPLOAD_ERR_OK){
$fileTmpPath=$_FILES['photo']['tmp_name'];
$fileName=$_FILES['photo']['name'];
$fileSize=$_FILES['photo']['size'];
$fileType=$_FILES['photo']['type'];
$uploadDir='uploads/';
$uploadFile=$uploadDir.uniqid().'_'.$fileName;
if(move_uploaded_file($fileTmpPath,$uploadFile)){
$photoUrl=$uploadFile;}else{
echo'Error uploading the file.';
exit();}}else{
$photoUrl=null;}
if($stmt->execute()){
echo'Book listing created successfully.';}else{
echo'Error creating book listing: '.$stmt->error;}
```

```
$stmt->close();
$conn->close();}
?><!DOCTYPE html>
<html>
<head>
  <title>Online Used Books Platform</title>
  <style>/* CSS styles */ </style>
</head><body><header>
    <h1>Online Used Books Platform</h1>
  </header>
  <nav>
    <ul>
          <li><a href="index.html">Home</a></li>
          <li><a href="search.html">Search</a></li>
          <li><a href="buy.html">Buy</a></li>
          <li><a href="sell.html">Sell</a></li>
          <li><a href="about.html">About</a></li>
    </ul>
  </nav>
  <main>
    <h2>Book Listing</h2>
    <form method="POST" enctype="multipart/form-data">
      <label for="title">Title</label>
      <input type="text" id="title" name="title" required>

      <label for="author">Author</label>
      <input type="text" id="author" name="author" required>

      <label for="description">Description</label>
      <textarea id="description" name="description" required></textarea>

      <label for="price">Price</label>
      <input type="text" id="price" name="price" required>

      <label for="condition">Condition</label>
        <select id="condition" name="condition1" required>
         <option value="">Select Condition</option>
         <option value="new">New</option>
         <option value="like-new">Like New</option>
         <option value="good">Good</option>
         <option value="fair">Fair</option>
          <option value="poor">Poor</option>
          </select>

      <label for="photo">Photo</label>
      <input type="file" id="photo" name="photo" accept="image/*" required>

      <button type="submit">Submit</button>
    </form>
</main></body></html>
```

- Search page:

- The code and the web page (see figure3) before the change:

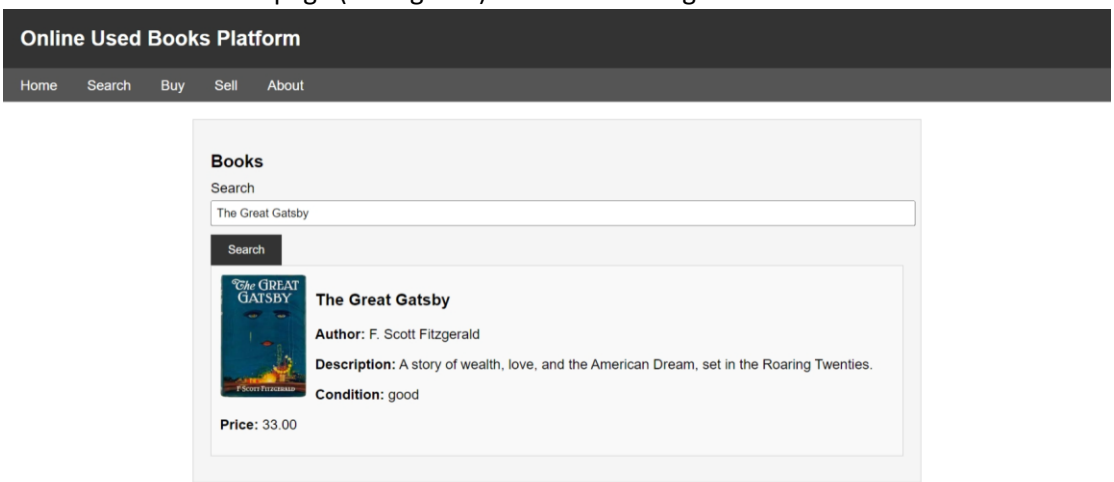


Figure 3: search page before change

```
<?php
$host = 'localhost';
$username = 'root';
$password = '';
$database = 'book';
$conn = new mysqli($host, $username, $password, $database);
if ($conn->connect_error) {
   die('Connection failed: ' . $conn->connect_error);}
$sql = "SELECT * FROM Books";
$result = $conn->query($sql);
$search = '';
if (isset($_GET['search'])) {
   $search = $_GET['search'];
   $sql = "SELECT * FROM Books WHERE title LIKE ?";
   $stmt = $conn->prepare($sql);
   $searchTerm = '%' . $search . '%';
   $stmt->bind_param("s", $searchTerm);
   $stmt->execute();
   $result = $stmt->get_result();
}
?>
<!DOCTYPE html>
<html>
<head>
   <title>Book Search</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         margin: 0;
         padding: 0;       }
      header {
         background-color: #333;
         color: #fff;
```

```
    padding: 20px;       }
h1 {
    margin: 0;
    font-size: 24px;       }
nav {
    background-color: #555;
    padding: 10px;       }
nav ul {
    list-style-type: none;
    margin: 0;
    padding: 0;       }
nav ul li {
    display: inline;
    margin-right: 10px;       }
nav ul li a {
    color: #fff;
    text-decoration: none;
    padding: 5px 10px;       }
nav ul li a:hover {
    background-color: #777;       }
main {
    max-width: 800px;
    margin: 20px auto;
    padding: 20px;
    background-color: #f5f5f5;
    border: 1px solid #ccc;       }
h2 {
    font-size: 20px;
    margin-bottom: 10px;       }
label {
    display: block;
    margin-bottom: 5px;       }
input[type="text"],
textarea {
    width: 100%;
    padding: 5px;
    margin-bottom: 10px;       }
button[type="submit"] {
    padding: 10px 20px;
    background-color: #333;
    color: #fff;
    border: none;
    cursor: pointer;       }
button[type="submit"]:hover {
    background-color: #555;       }
.book-listing {
    border: 1px solid #ccc;
    padding: 10px;
    margin-bottom: 10px;
    background-color: #f9f9f9;       }
.book-listing img {
```

```
        max-width: 100px;
        max-height: 150px;
        margin-right: 10px;
        float: left;       }
  </style>
</head><body>   <header>       <h1>Online Used Books Platform</h1>   </header>
  <nav>
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">Search</a></li>
      <li><a href="#">Buy</a></li>
      <li><a href="#">Sell</a></li>
      <li><a href="#">About</a></li>
    </ul>
  </nav>   <main>
    <h2>Books</h2>
    <form method="GET">
      <label for="search">Search</label>
      <input type="text" id="search" name="search" value="<?php echo
htmlspecialchars($search); ?>">
      <button type="submit">Search</button>
    </form>
    <?php
    if ($result->num_rows > 0) {
      while ($row = $result->fetch_assoc()) {
        echo '<div class="book-listing">';
        echo '<img src="' . $row['photo_url'] . '" alt="Book Photo">';
        echo '<h3>' . $row['title'] . '</h3>';
        echo '<p><strong>Author:</strong> ' . $row['author'] . '</p>';
        echo '<p><strong>Description:</strong> ' . $row['description'] . '</p>';
        echo '<p><strong>Condition:</strong> ' . $row['condition1'] . '</p>';
        echo '<p><strong>Price:</strong> ' . $row['price'] . '</p>';
        echo '</div>';          }
    } else {
      echo 'No books found.';       }
    ?>
  </main>
</body>
</html>
```

- The code and the web page (figure 5,6) after the change:

```
<?php
$host = 'localhost';
$username = 'root';
$password = '';
$database = 'book';
$conn = new mysqli($host, $username, $password, $database);
if ($conn->connect_error) {
   die('Connection failed: ' . $conn->connect_error);}
$sql = "SELECT * FROM Books";
$result = $conn->query($sql);
$search = '';
if (isset($_GET['search'])) {
   $search = $_GET['search'];
   $sql = "SELECT * FROM Books WHERE title LIKE ? OR author LIKE ? OR description LIKE
?";    $stmt = $conn->prepare($sql);
   $searchTerm = '%' . $search . '%';
   $stmt->bind_param("sss", $searchTerm, $searchTerm, $searchTerm);
   $stmt->execute();
   $result = $stmt->get_result();
}
?>
<!DOCTYPE html>
<html>
<head>
   <title>Book Search</title>
   <style>/* CSS styles */    </style>
</head><body>
      <?php
      //same code above
      ?>
   </main>
</body>
</html>
```

Testing:

Testing is conducted to verify that the changes introduced in the CR have been applied successfully and are functioning as intended.

Test Case	TC-001
Test Case Description	Verify that a user can successfully create a book listing with all the required information.
Precondition	- The user is logged into the online used books platform. - The user wants to sell a book and has all the necessary information (title, photo, price, description, condition, and author).
Test Steps	1. Click on the "Sell"in the header. 2. Enter the title "The Great Gatsby" in the title field. 3. Upload a suitable photo of the book. 4. Enter the price as 33. 5. Enter a brief description of the book. 6. Select the book condition as "Good." 7. Enter the author as "F. Scott Fitzgerald." 8. Click on the "Submit" button. 9. Navigate to the user's book listings page.
Expected Result	-The system should successfully create a book listing with all the provided information. -The book "The Great Gatsby" by F. Scott Fitzgerald should be listed for sale on the user's book listings page. -The book listing should include the uploaded photo, price, description, condition, and author information as entered by the user.
Actual Result	See (Figure 4).
Pass/Fail	Pass

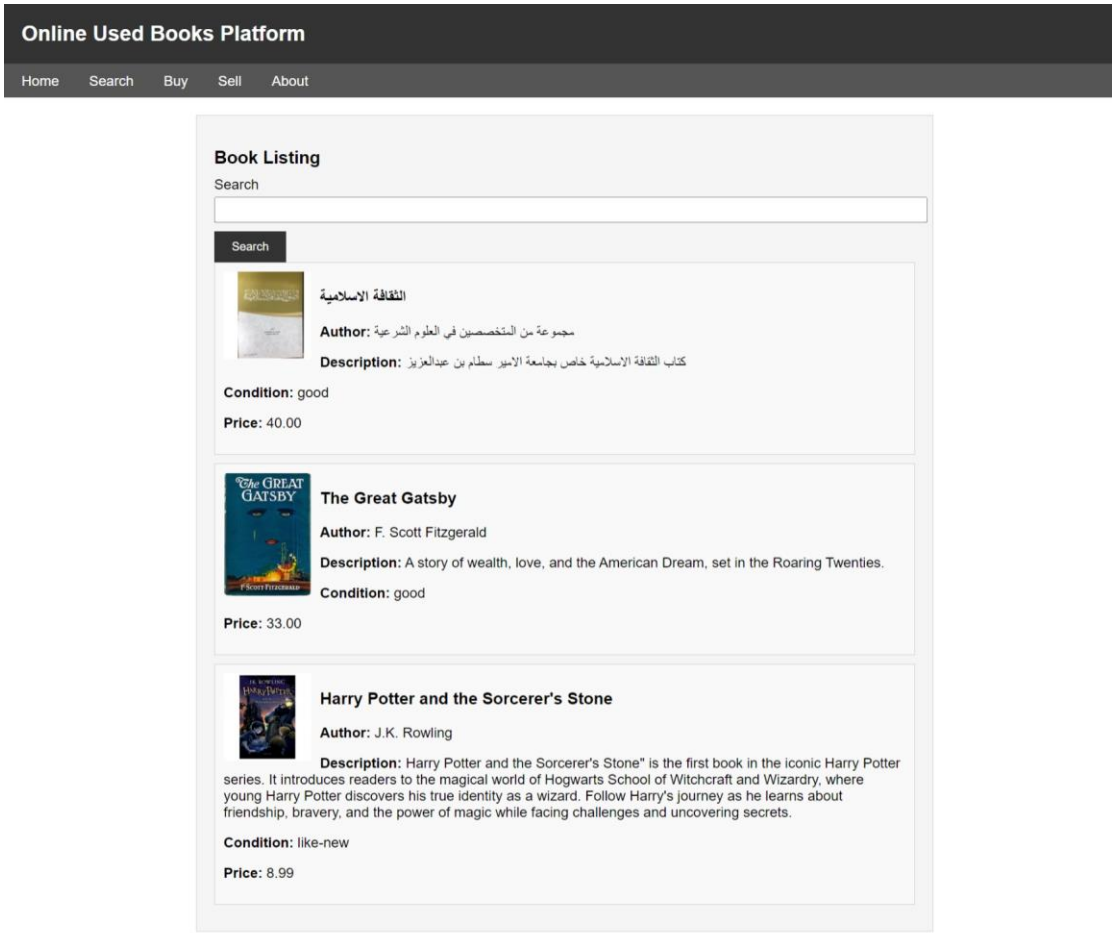


Figure 4: book listing after adding a book

Test Case	TC-002
Test Case Description	Verify that the system displays accurate search results when a user searches for a book by author name.
Precondition	- The user is logged into the online used books platform. - The system contains books with various titles and authors.
Test Steps	1. Enter the author name " J.K. Rowling " in the search bar. 2. Click on the search button.
Expected Result	The system should accurately display search results that match the entered author name, including the book "Harry Potter and the Sorcerer's Stone" by J.K. Rowling.
Actual Result	See (figure 5)
Pass/Fail	Pass

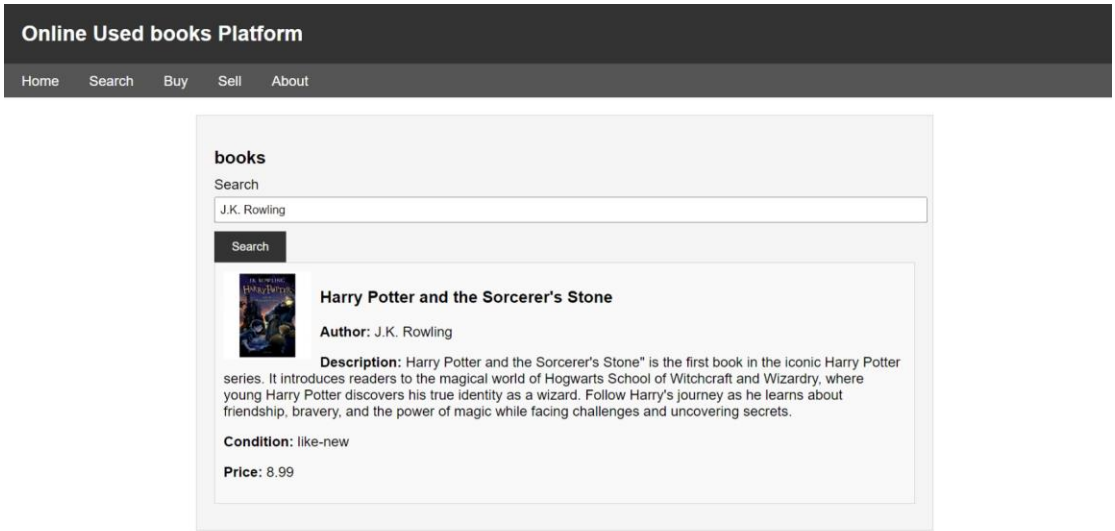


Figure 5: search page after change 1

Test Case	TC-003
Test Case Description	Verify that the system displays accurate search results when a user searches for a book by description.
Precondition	- The user is logged into the online used books platform. - The system contains books with various titles and authors.
Test Steps	1. Enter the description "كتاب الثقافة الاسلامية خاص بجامعة الامير سطام بن عبدالعزيز " in the search bar. 2. Click on the search button.
Expected Result	The system should accurately display search results that match the entered description, including the book
Actual Result	See (figure 6)
Pass/Fail	Pass

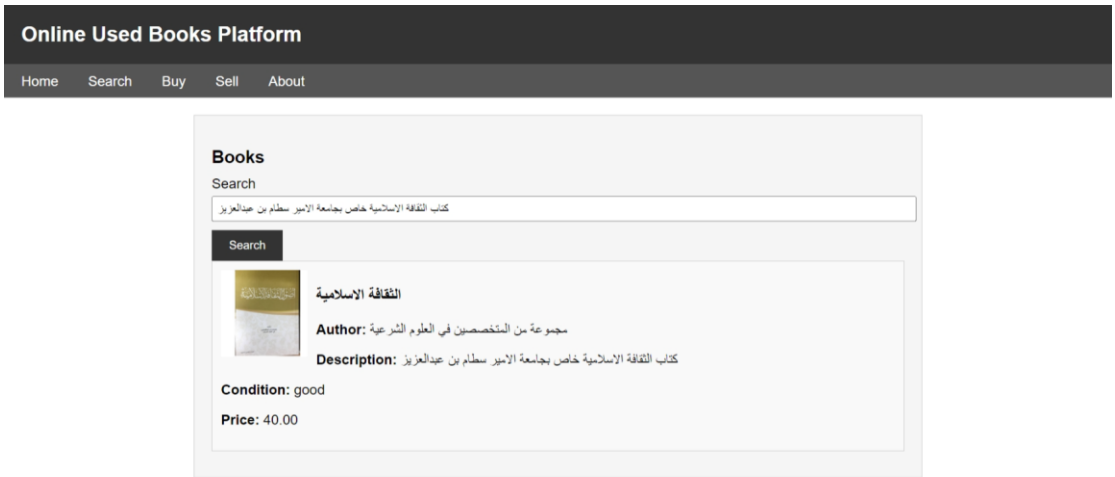


Figure 6: search page after change 2

Project Description after the change:

Functional Requirements

1. Register:
   - REQ-1: The user shall enter their email.
   - REQ-2: The user shall enter their username.
   - REQ-3: The user shall enter their phone number.
   - REQ-4: The system shall validate that the entered email and phone number are not already registered.
   - REQ-5: The user shall enter a password.

2. Log in:
   - REQ-1: The user shall enter their email or username.
   - REQ-2: The user shall enter their password.
   - REQ-3: The user shall be able to request a password reset.

3. Searching for a book:
   - REQ-1: The user shall enter the search criteria such as author name, book description, and book condition in the search bar.
   - REQ-2: The system shall display search results based on the entered information.
   - REQ-3: If there are no exact matches, the system shall display a notification informing the user.

4. Buying a book:
   - REQ-1: The user shall select the book(s) they want to buy.
   - REQ-2: The system shall show the buyer of the seller's approval or rejection of the purchase process.

5. Payment:
   - REQ-1: The user shall enter their location for shipping.
   - REQ-2: The user shall enter their credit card information for payment.

6. Displaying a book for sale:
   - REQ-1: The user shall enter the title of the book.
   - REQ-2: The user shall upload a photo of the book.
   - REQ-3: The user shall enter the price of the book.
   - REQ-4: The user shall enter the description of the book.
   - REQ-3: The user shall enter the condition (new-like new-fair-good-bad) of the book.
   - REQ-3: The user shall enter the author of the book.

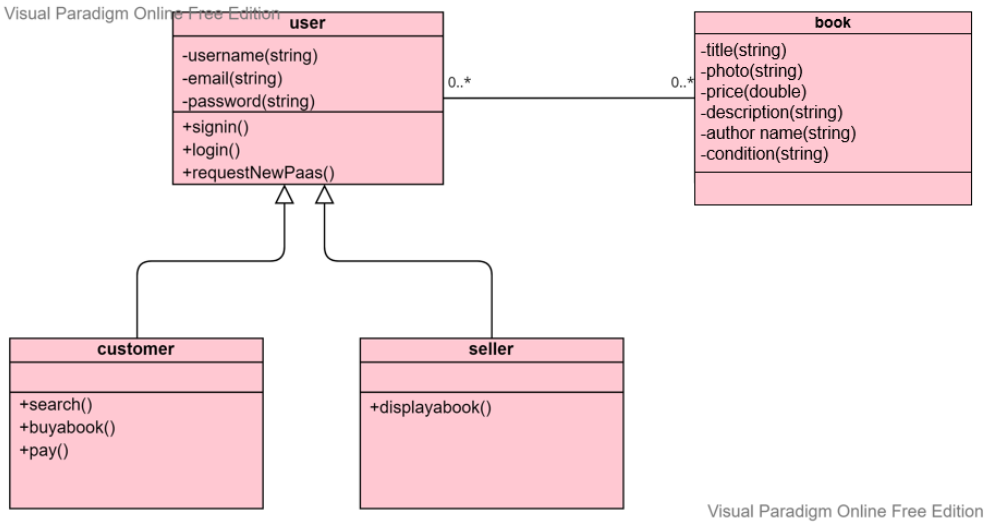
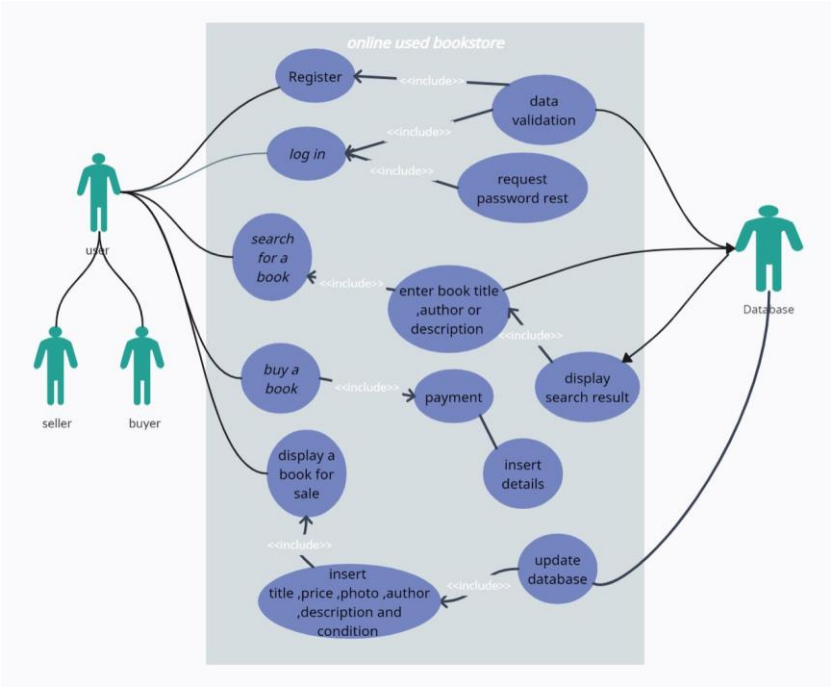
7. Purchase order status:
   - REQ-1: The system shall send a notification to the buyer's email when someone requests to buy the book.
   - REQ-2: The seller shall go to the platform to determine the status of the request.

Non-functional requirements:

There are no changes.

Diagrams:

- Class diagram:



- Use case :